

Application X

Realizat de:

Roman Andrei

2024

CUPRINS

I.	MOTIVAȚIA ALEGERII TEMEI.....	3
II.	DESCRIEREA APLICAȚIEI.....	4 -15
III.	DESCRIEREA TEHNOLOGIEI FOLOSITE.....	16 -24
IV.	BIBLIOGRAFIE.....	25 -26

I. MOTIVAȚIA ALEGERII TEMEI

În era digitală în care trăim, tehnologia este motorul ce redefinește modalitățile în care interacționăm cu lumea din jurul nostru. Inspirat de această realitate dinamică, am ales să dezvolt o aplicație de catalog școlar, ca temă a atestatului meu de finalizare a studiilor. Motivul fundamental al alegerii acestei teme rezidă în potențialul său de a aduce beneficii palpabile atât în cadrul comunității școlare, cât și în dezvoltarea mea personală și profesională.

În primul rând, o aplicație de catalog școlar reprezintă un instrument esențial pentru eficientizarea proceselor educaționale. Prin intermediul ei, elevii, părinții și cadrele didactice pot accesa rapid și ușor informații despre note, absențe sau alte aspecte relevante pentru parcursul școlar al fiecărui elev. Astfel, se creează o punte de comunicare directă și transparentă între toate părțile implicate în procesul educațional, contribuind la creșterea eficienței acestuia.

În al doilea rând, dezvoltarea unei aplicații de acest gen reprezintă pentru mine o oportunitate de a-mi pune în valoare abilitățile și cunoștințele în domeniul tehnologic. Prin programarea și designul acestei aplicații, voi avea șansa de a exersa și îmbunătăți competențele mele în programare, dar și în gestionarea proiectelor și rezolvarea problemelor practice. Aceste aptitudini nu doar că îmi vor consolida baza de cunoștințe, dar vor reprezenta și un avantaj considerabil în viitoarea mea carieră profesională, într-un domeniu care e într-o continuă evoluție și cerere. Prin crearea unei aplicații care simplifică și eficientizează gestionarea informațiilor școlare, nu doar că îmi propun să aduc o contribuție concretă în comunitatea școlară, dar și să încurajez o viziune progresistă asupra educației. Astfel, fiecare linie de cod și fiecare element de design devin părți ale unei povesti care subliniază importanța evoluției tehnologice în contextul educațional și aspirația noastră comună către un viitor mai conectat și mai eficient.

În concluzie, tema aleasă pentru atestatul meu nu este doar o sarcină academică, ci și o oportunitate de a contribui la îmbunătățirea sistemului educațional și de a-mi dezvolta aptitudinile într-un mod practic și relevant. Sunt convins că prin angajamentul și pasiunea mea pentru acest proiect, voi reuși să aduc o contribuție semnificativă în mediul școlar și să îmi conturez un viitor profesional solid în domeniul tehnologiei.

II. DESCRIEREA APLICAȚIEI

A. Introducere

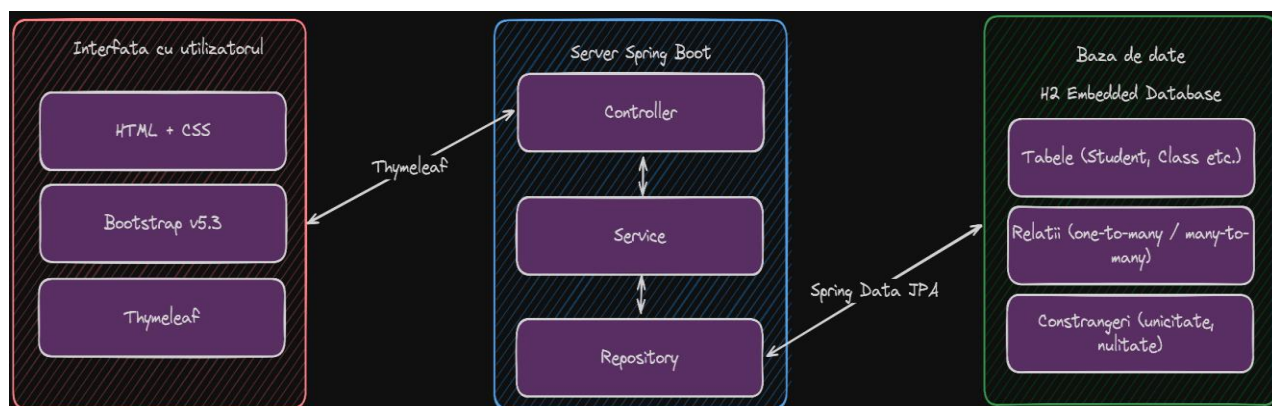
Application X ajută la crearea și stocarea mult mai ingenioasă a datelor unui elev, ale unui profesor, ale claselor, dar și ale materiilor, față de un catalog obișnuit. Aplicația poate stoca și procesa date despre: Teacher, Classes, Students, Grades, Subjects. În cadrul fiecărei părți poate să fie accesată o pagină în care se poate vedea o listă, sub forma tabelară a tuturor datelor pe care le are orice profesor, clasa, elevi și materii, dar și o pagină unde se pot adăuga profesori noi, elevi noi, materii noi, dar și clase noi, bineînțeles și note.

Aplicația ne pune la dispoziție un tablou de control unde putem vedea lista tuturor claselor. Dacă accesăm o clasă, putem vedea o listă cu toți elevii și cu toți profesorii care predau la acea clasă. Iar dacă accesăm un elev putem vedea notele lui și putem adăuga note suplimentare.

Link către sistemul de versionare online folosit (GitHub):

https://github.com/Ninjutzu24/school_management

B. Design-ul sistemului



În diagrama de mai sus regăsim design-ul întregului sistem. Acesta este compus din trei componente mari: baza de date, serverul și interfața cu utilizatorul. În acest mod, putem urmări cu ușurință cum decurge procesarea datelor și ce componente interne folosește fiecare din aceste componente majore.

Baza de date pe care am folosit-o este H2 Embedded Database, pentru ușurința în utilizare, deoarece nu trebuie să instalăm practic nicio bază de date, ci ea se deschide odată cu serverul nostru de Java. În această componentă, se persistă datele, sub formă tabelară.

Fiind o baza de date relațională, vor exista și relații între tabele, dar și anumite constrângeri pentru fiecare tabel sau câmp.

În continuare, o altă componentă majoră este serverul de Spring Boot. Acesta este scris utilizând limbajul de programare Java și framework-ul Spring Boot. Pentru a separa funcționalitățile am creat trei straturi diferite: Controller, Service și Repository.

Repository este stratul care se ocupă cu comunicarea cu baza de date, prin intermediul librăriei Spring Data JPA. O componentă adiacentă în acest sens este și pachetul de model, unde avem definite toate tabelele din baza de date sub forma unor entități.

Stratul de Service este unul intermediar, având ca rol posibilitatea adăugării unor reguli, validări și altă logică suplimentară între straturile care au ca rol comunicarea cu baza de date, respectiv comunicarea cu interfața cu utilizatorul.

Nu în ultimul rând, stratul Controller este cel care definește căile pe care utilizatorul poate naviga (link-urile) și resursele pe care le poate accesa. Acesta conține endpoint-urile disponibile, tipul lor (GET, POST) și căile de acces către fișierele HTML. Tot această componentă comunică și cu interfața cu utilizatorul, prin intermediul librăriei Thymeleaf.

În final, componenta interfeței cu utilizatorul folosește elemente ale limbajelor de programare HTML, CSS, precum și o librărie populară în rândul dezvoltatorilor de pagini web, numită Bootstrap, care ne pune la dispoziție multe componente, stiluri, layout-uri și altele. De asemenea, tot în această componentă folosim și componente interne ale librăriei Thymeleaf, prin intermediul cărora accesăm date din serverul Spring Boot, sau trimitem date către același server.

C. Baza de date

Am realizat mai multe tabele care au între ele mai multe relații. Fiecare tabelă conține o cheie primară, numită “id”, de asemenea două câmpuri care se numesc “created_at” și “updated_at” prin care putem ține evidența momentului creării și al actualizării datelor.

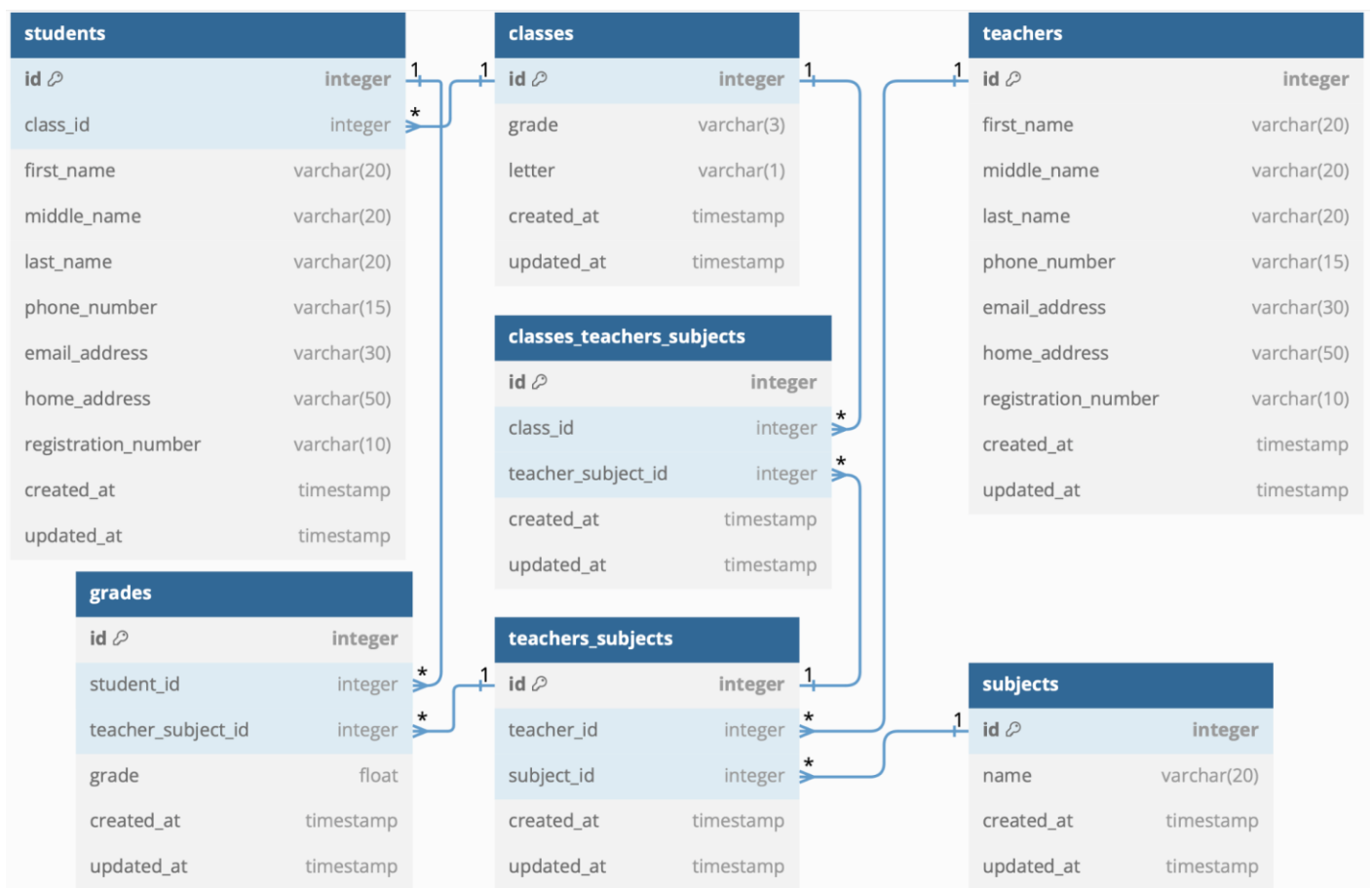
Tabela denumită “**students**” conține: first_name, middle_name, last_name, phone_number, email_address, home_address și registration_number. Cu ajutorul acestei tabele putem să memorăm informațiile esențiale ale fiecărui elev.

Tabela “**classes**” conține: grade și letter. Cu ajutorul acestei tabele putem să memorăm informațiile esențiale ale fiecărei clase (numărul clasei, dar și cum se numește clasa respectivă)

Tabela “**teachers**” conține: first_name, middle_name, last_name, phone_number, email_address, home_address si registration_nmber. Prin aceasta tabela putem memora informațiile esențiale ale unui profesor (referitoare la numele lui, prenumele lui, numărul de telefon, etc)

Tabela “**grades**” conține: gradeValue. Prin aceasta tabela putem memora informațiile referitoare la notele fiecărui elev.

Tabela “**subjects**” conține: name. Prin aceasta tabela putem memora informațiile referitoare la cum se numește materia respectivă.



Relațiile dintre tabele sunt evidențiate prin conexiuni ce au ca elemente o linie ce poate avea la capăt valorile 0, 1 sau “ * ”. Relațiile 1...” * ” denota o relație de one-to-many, iar relațiile *....* denota o relație many-to-many. Pentru fiecare relație am avut nevoie de anumite câmpuri numite chei secundare (foreign key).

În acest mod am putut obține o structura flexibila, în care un profesor poate preda la mai multe materii și în același timp la mai multe clase. De asemenea, un student aparține unei singure clase, dar poate primi mai multe note, la diverse materii date de anumiți profesori.

D. Interfata (HTML, CSS, Thymeleaf) si Serverul Spring Boot (Java)

În cadrul proiectului am utilizat o componenta de server Spring Boot, cu ajutorul limbajului de programare Java. In continuare, voi prezenta un parcurs complet al procesarii datelor, de la conexiunea cu baza de date, pana la afisarea si obtinerea datelor din interfata cu utilizatorul. Exemplul de mai jos afiseaza lista tuturor studentilor din scoala.

Fisierul students/students.html, contine HTML, CSS si elemente ale bibliotecii Thymeleaf. In acest fisier, am importat o librarie ce ne ajuta prin componente usor de folosit - Bootstrap. In partea de body, avem ca prim element un navigation bar, ce contine tot meniul aplicatiei. In continuare afisam un tabel in care expunem detaliile fiecarui student pe cate un rand. Ceea ce face posibil acest lucru, este folosirea atributului “students”, care este primit din StudentController.java. Folosind sintaxa “th:each”, creăm un iterator care parcurge toti studentii, iar pentru fiecare student se afiseaza tabelar, pe cate o linie, detaliile acestuia. In ultima coloana, vom avea un buton, corespunzator studentului de pe linia respectiva, prin care putem ajunge pe o pagina detaliata despre acel student.

```
<!DOCTYPE html>

<html lang="en" xmlns:th="http://www.thymeleaf.org">

<head>

  <meta charset="utf-8">

  <meta name="viewport" content="width=device-width, initial-scale=1">

  <title>Application X</title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"

        integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
        crossorigin="anonymous">

</head>

<body style="background-color: dimgray">

<nav class="navbar navbar-dark bg-dark fixed-top">

  <div class="container-fluid">

    <a class="navbar-brand" href="#">School Management Tool</a>

    <button      class="navbar-toggler"      type="button"      data-bs-toggle="offcanvas"      data-bs-
target="#offcanvasDarkNavbar"

      aria-controls="offcanvasDarkNavbar" aria-label="Toggle navigation">

      <span class="navbar-toggler-icon"></span>
```

```

</button>

<div class="offcanvas offcanvas-end text-bg-dark" tabindex="-1" id="offcanvasDarkNavbar"
    aria-labelledby="offcanvasDarkNavbarLabel">

    <div class="offcanvas-header">

        <h5 class="offcanvas-title" id="offcanvasDarkNavbarLabel">Menu</h5>

        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="offcanvas"
            aria-label="Close"></button>

    </div>

    <div class="offcanvas-body">

        <ul class="navbar-nav justify-content-end flex-grow-1 pe-3">

            <li class="nav-item"><a class="nav-link" href="http://localhost:8080">Home</a></li>

            <!-- Teachers -->

            <li class="nav-item dropdown">

                <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown"
                    aria-expanded="false">Teachers</a>

                <ul class="dropdown-menu dropdown-menu-dark">

                    <li><a class="dropdown-item" href="http://localhost:8080/teachers">Teachers List</a></li>

                    <li><hr class="dropdown-divider"></li>

                    <li><a class="dropdown-item" href="http://localhost:8080/teacher/form">Add a
teacher</a></li>

                    <li><hr class="dropdown-divider"></li>

                    <li><a class="dropdown-item" href="http://localhost:8080/teacher/subject/form">Assign a
Subject to Teacher</a></li>

                </ul>

            </li>

            <!-- Students -->

            <li class="nav-item dropdown active">

                <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown"
                    aria-expanded="false">Students</a>

                <ul class="dropdown-menu dropdown-menu-dark">

                    <li><a class="dropdown-item active" aria-current="page"
href="http://localhost:8080/students">Students List</a></li>

                    <li><hr class="dropdown-divider"></li>

                    <li><a class="dropdown-item" href="http://localhost:8080/student/form">Add a
student</a></li>

```



```

        </ul>

    </li>

    <!-- Classes -->

    <li class="nav-item dropdown">

        <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown"
            aria-expanded="false">Classes</a>

        <ul class="dropdown-menu dropdown-menu-dark">

            <li><a class="dropdown-item" href="http://localhost:8080/classes">Classes List</a></li>

            <li><hr class="dropdown-divider"></li>

            <li><a class="dropdown-item" href="http://localhost:8080/class/form">Add a class</a></li>

            <li><hr class="dropdown-divider"></li>

            <li><a class="dropdown-item" href="http://localhost:8080/class/teacher/form">Add a Teacher
to a Class</a></li>

        </ul>

    </li>

    <!-- Subjects -->

    <li class="nav-item dropdown active">

        <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown"
            aria-expanded="false">Subjects</a>

        <ul class="dropdown-menu dropdown-menu-dark">

            <li><a class="dropdown-item" href="http://localhost:8080/subjects">Subjects List</a></li>

            <li><hr class="dropdown-divider"></li>

            <li><a class="dropdown-item" href="http://localhost:8080/subject/form">Add
subject</a></li>

        </ul>

    </li>

    <!-- Grades -->

    <li class="nav-item dropdown">

        <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown"
            aria-expanded="false">Grades</a>

        <ul class="dropdown-menu dropdown-menu-dark">

            <li><a class="dropdown-item" href="http://localhost:8080/grades">Grades List</a></li>

            <li><hr class="dropdown-divider"></li>

            <li><a class="dropdown-item" href="http://localhost:8080/grade/form">Add a grade</a></li>

```

```

        </ul>

    </li>

</ul>

</div>

</div>

</div>

</div>

</nav>

<h1 style="text-align: center; color: white; margin-bottom: 20px; margin-top: 60px">Students</h1>

<table class="table table-dark">

    <thead>

        <th scope="col">ID</th>

        <th scope="col">Class</th>

        <th scope="col">First Name</th>

        <th scope="col">Middle Name</th>

        <th scope="col">Last Name</th>

        <th scope="col">Phone Number</th>

        <th scope="col">Email Address</th>

        <th scope="col">Home Address</th>

        <th scope="col">Registration Number</th>

        <th scope="col"></th>

    </thead>

    <tbody>

        <tr th:each="student : ${students}">

            <td th:text="${student.id}"></td>

            <td th:text="${student.clazz != null ? (student.clazz.grade + student.clazz.letter) : '-'}"></td>

            <td th:text="${student.firstName}"></td>

            <td th:text="${student.middleName}"></td>

            <td th:text="${student.lastName}"></td>

            <td th:text="${student.phoneNumber}"></td>

            <td th:text="${student.emailAddress}"></td>

            <td th:text="${student.homeAddress}"></td>

            <td th:text="${student.registrationNumber}"></td>

            <td><a class="btn btn-primary" th:href="${'http://localhost:8080/student/' + student.id}">More
Details</a></td>

```

```

</tr>

</tbody>

</table>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
    crossorigin="anonymous"></script>

</body>

</html>

```

În continuare voi prezenta fișierul StudentController.java. Pentru simplitate, am păstrat doar partea de cod care interacționează cu fișierul HTML de mai sus. Aceasta clasa java este responsabilă cu interacțiunea între interfața cu utilizatorul și serverul Spring Boot. Aici avem endpoint-ul GET “/students” prin care solicităm mai departe layer-ului de Service, datele asupra tuturor studenților din sistem. Toate aceste date, le setăm prin parametrul Model, care știe să comunice cu interfața HTML, prin adăugarea atributului “students”. Metoda trebuie să returneze calea către fișierul HTML care se dorește a fi afișat, în acest caz: “students/students”. De precizat, este ca această clasă are și două câmpuri importante importate automat datorită adnotării @RequiredArgsConstructor: ClazzService și StudentService, cea din urmă fiind detaliată mai jos.

```

package edu.cdloga.school_management.controller;

import edu.cdloga.school_management.model.Clazz;
import edu.cdloga.school_management.model.Student;
import edu.cdloga.school_management.service.ClazzService;
import edu.cdloga.school_management.service.StudentService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.util.Comparator;
import java.util.stream.Collectors;

import static java.util.Comparator.comparing;

```

```

@Controller

@RequiredArgsConstructor

public class StudentController {

    private final StudentService studentService;

    private final ClazzService clazzService;

    @GetMapping(path = "/students")

    public String getStudents(Model model) {

        model.addAttribute("students", studentService.getAllStudents());

        return "students/students";

    }

    .....

}

```

Clasa StudentService.java este pur si simplu un layer care conține logica de business a proiectului. În cazul nostru, pentru a păstra simplitatea, aceasta clasă are doar o metodă care apelează următorul layer, de repository - care la rândul lui știe să comunice cu baza de date, nefiind necesar sa procesăm datele suplimentar.

```

package edu.cdloga.school_management.service;

import edu.cdloga.school_management.model.Student;

import edu.cdloga.school_management.repository.StudentRepository;

import lombok.RequiredArgsConstructor;

import org.springframework.stereotype.Component;

import java.util.Optional;

import java.util.Set;

@Component

@RequiredArgsConstructor

public class StudentService {

    private final StudentRepository studentRepository;

```

```

public Set<Student> getAllStudents() {

    return studentRepository.getAllStudents();

}

.....

}

```

Interfața StudentRepository.java este cea care știe să comunice cu baza de date, prin intermediul librăriei Spring Data JPA. Aceasta extinde interfața CrudRepository, care ne pune la dispoziție în mod automat anumite funcții pe care le putem folosi pentru a manipula date din baza de date, precum: findById, save și altele. De asemenea, dacă se dorește, se pot adăuga interogări native, așa cum am făcut și eu, pentru a putea extrage toți studenții din baza de date.

```

package edu.cdloga.school_management.repository;

import edu.cdloga.school_management.model.Student;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Component;

import java.util.Set;

@Component

public interface StudentRepository extends CrudRepository<Student, Long> {

    @Query(
        value = "select * from students;",
        nativeQuery = true
    )

    Set<Student> getAllStudents();

}

```

În final, este important să prezint și clasa Student.java, care se află în pachetul model. Această clasă este foarte importantă, ea este creată cu ajutorul adnotărilor din librăria Spring Data JPA și reprezintă o entitate (o tabelă) din baza de date. Practic, aici am definit fiecare câmp din tabela “students”, de asemenea și toate relațiile ale tabelului cu alte tabele - spre exemplu, cu tabela “classes”. Adnotările @Getter și @Setter, sunt posibile datorită librăriei Lombok, și ne ajută în generarea automată a tuturor metodelor de get și set pe câmpurile clasei noastre. De asemenea, pentru fiecare câmp în parte, se pot defini anumite constrângeri, cum ar fi constrângerea ca acel câmp să nu fie null, sau ca acel câmp să fie unic în baza de date. De asemenea, tot aici am definit și cheia primară (primary key), cu precizarea ca lăsăm la latitudinea bazei de date să genereze automat aceste id-uri.

```
package edu.cdloga.school_management.model;
```

```
import jakarta.persistence.*;
```

```
import lombok.Getter;
```

```
import lombok.Setter;
```

```
import org.hibernate.annotations.CreationTimestamp;
```

```
import org.hibernate.annotations.UpdateTimestamp;
```

```
import java.time.LocalDateTime;
```

```
import java.util.Set;
```

```
@Getter
```

```
@Setter
```

```
@Entity
```

```
@Table(name = "students")
```

```
public class Student {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
@ManyToOne
```

```
@JoinColumn(name = "class_id")
```

```
private Clazz clazz;
```

```
@OneToMany(mappedBy = "student")
```

```
private Set<Grade> grades;
```

```
@Column(nullable = false)
```

```
private String firstName;
```

```
@Column(nullable = true)
```

```
private String middleName;
```

```
@Column(nullable = false)
```

```
private String lastName;
```

```
@Column(nullable = false)
```

```
private String phoneNumber;
```

```
@Column(nullable = false)
```

```
private String emailAddress;
```

```
@Column(nullable = false)
```

```
private String homeAddress;
```

```
@Column(nullable = false, unique = true)
```

```
private String registrationNumber;
```

```
@CreationTimestamp
```

```
@Column(nullable = false, updatable = false)
```

```
private LocalDateTime createdAt;
```

```

@UpdateTimestamp
@Column(nullable = false)
private LocalDateTime updatedAt;

}

```

III. DESCRIEREA TEHNOLOGIEI FOLOSITE

În realizarea aplicației am folosit diverse mijloace de memorare a datelor, de a prelucra datele și diferite biblioteci, pentru a ne ajuta la crearea acesteia. În structura de mai jos sunt explicate tehnologiile folosite și ce reprezintă fiecare, dar este scoasă în evidență și profunzimea importantă a acestora.

A. Baze de date

Bazele de date sunt mijloace de stocare a informațiilor și a datelor pe un suport extern prin care ne este conferit un acces ușor și rapid asupra componentelor acestora. Acestea sunt de două feluri, Baze de date relaționale și Baze de date nerelaționale (NoSQL). *Bazele de date relaționale* conțin elemente care sunt organizate ca un set de tabele, cu rânduri și coloane. Acest tip de baze de date reprezintă cel mai eficient și flexibil mod de a accesa informații structurale. *Bazele de date nerelaționale* permit stocarea și gestionarea datelor nestructurate. Atunci când vorbim de baze de date cel mai important lucru este identificăm ce tip de bază de date trebuie să folosim, relațională sau nerelațională. Ambele structuri de baze de date oferă avantaje unice și se folosesc pentru necesități diferite. Bazele de date NoSQL oferă o structură mult mai flexibilă și reține date dinamice sau nestructurate, în timp ce bazele de date SQL folosesc limbaje de interogări structurate și au scheme predefinite. Eu am ales să folosesc o bază de date de tip relațională, pentru care este necesară cunoașterea limbajului de programare SQL.

1. H2 Embedded Database

H2 Embedded Database este un sistem relațional de management al bazelor de date scrise în Java. Este o aplicație de tipul client/server care stochează în memorie, nu persista datele pe un disk. Feature-urile ale H2 Database:

- foarte rapidă și open-source API
- disk-based sau in-memory baze de date
- Aplicații de tip browser-based Console
- Căutări de tip Fulltext

2. Limbajul SQL

Limbajul SQL este un limbaj standard pentru accesarea bazelor de date relaționale prin intermediul căruia putem să descriem și să manipulăm date. Majoritatea sistemelor de gestionare a bazelor de date relaționale utilizează instrucțiuni ale limbajului SQL. Câteva instrucțiuni specifice acestui limbaj sunt următoarele: SELECT, FROM, WHERE, SET

B. Java

Java este un limbaj de programare orientat pe obiecte, conceput de către James Gosling la începutul anilor '90. Acesta a fost lansat ulterior în anul 1995. Cele mai multe aplicații distribuite au fost codate în Java. În felul acesta se creează o platformă unică, la nivelul programatorului, deasupra unui mediu extrem de diversificat. Acest limbaj a împrumutat mare parte din sintaxa C și C++, dar prezintă un model al obiectelor mult mai simplu. Un fișier Java, corect scris, poate să ruleze fără modificări pe orice platformă care are instalată o mașină virtuală Java, acest lucru fiind posibil, deoarece sursele Java sunt compilate într-un mod standard, numit Cod de octeți. Mașina virtuală Java este mediul în care se execută programarea Java.

1. Programare orientată pe obiecte

Programarea orientată pe obiecte este unul din cei mai importanți pași făcuți în evoluția limbajelor de programare spre o mai puternică abstractizare în implementarea programelor. Aceasta a apărut din necesitatea exprimării unei probleme într-un mod cât mai simplu, pentru a putea fi înțeleasă de cât mai mulți programatori. Astfel părțile de cod ce alcătuiesc un program se apropie mai mult de modul nostru de a gândi decât de modul de lucru al unui calculator. Până la apariția programării orientate pe obiect, programele erau implementate în limbaje de programare procedurale (C, Pascal) sau în limbaje care nici măcar nu ofereau o modalitate de grupare a instrucțiunilor în unități logice, cum este cazul limbajului de asamblare (assembler). Deoarece, în trecut problema programării procedurale era separarea datelor de unitățile care prelucrau datele, s-a format conceptul de **clasa**. Clasa grupează datele și unitățile de prelucrare a acestora într-un modul, unindu-se astfel într-o entitate mult mai naturală. Conceptul de bază al programării pe obiecte este Clasa. Un exemplu concret care ne va ajuta să înțelegem mai bine conceptul de Programare pe orientată pe obiecte, vom folosi exemplul următor:

Clasa “floare” desemnează toate plantele care au flori, iar clasa “Fruct” desemnează toate obiectele pe care noi le identificăm ca fiind fructe.

C. Spring

Spring este un Framework ce oferă un model cuprinzător de programare și configurare pentru aplicații de întreprindere bazate pe Java pe orice platforma de implementare. Spring se concentrează pe „plumbing”-ul aplicațiilor, astfel încât să nu mai trebuiască să ne concentrăm atât de mult pe nivelul de aplicație, fără legături inutile cu mediile de implementare specifice. Spring framework oferă o mulțime de avantaje aplicației: creează o metoda Java care rulează în cadrul bazelor de date fără ajutorul API-urilor, creează proceduri locale care definesc proceduri de acces, fără ajutorul API-urilor. Prescurtarea de **API** vine de la “Application Programming Interface”.

1. Boot

Spring Boot este un aliat indispensabil pentru dezvoltatorii dornici să creeze aplicații moderne, scalabile și eficiente. Prin abordarea sa inovatoare și simplificarea procesului de dezvoltare, Spring Boot redefinește modul în care aplicațiile Java sunt create și gestionate în ziua de azi. La baza, Spring Boot se bazează pe framework-ul de dezvoltare Spring, însă aduce cu sine un set de funcționalități care transformă experiența celor ce o folosesc. Prin simplificarea configurării și automatizarea sarcinilor repetitive, Spring Boot elimină barierele inițiale din calea dezvoltării unei aplicații, permițând programatorilor să se concentreze mai mult pe logica aplicației și mai puțin pe detaliile tehnice.

Unul dintre aspectele remarcabile ale Spring Boot este capacitatea sa de a *accelera dezvoltarea prin oferirea unui set de configurații implicite și integrarea simplă cu alte tehnologii*. Acest lucru permite celor ce o folosesc să creeze rapid prototipuri și să lanseze aplicații Java de înaltă calitate, în timp ce se pot baza pe suportul unei comunități active și a unei documentații bogate.

Deci, Spring Boot nu este doar un cadru de dezvoltare, ci și o manifestare a spiritului inovator și progresist al comunității de dezvoltatori Java. Prin simplificarea procesului de dezvoltare și accelerarea ciclului de livrare a aplicațiilor, Spring Boot devine un instrument esențial pentru orice dezvoltator care aspiră la excelență în crearea de aplicații Java moderne și eficiente.

2. Data JPA

Data JPA reprezintă un aspect esențial în dezvoltarea aplicațiilor Java bazate pe baze de date relaționale. Ca parte a ecosistemului Spring, Data JPA oferă un mod simplificat și elegant de a interacționa cu bazele de date prin intermediul limbajului de programare Java.

La baza, JPA (Java Persistence API) este o specificație Java care definește o metodă standard pentru a realiza mapping-ul obiect-relațional (ORM), permițând dezvoltatorilor să modeleze datele din bazele de date relaționale utilizând obiecte Java. Data JPA, în particular, este o implementare a acestei specificații de către Spring, care *integrează funcționalități suplimentare pentru a face interacțiunea cu bazele de date mai ușoară și mai intuitivă.*

Principalele beneficii ale utilizării Data JPA includ:

-> Abstracția complexității SQL: Data JPA abstractizează detalii tehnice legate de limbajul SQL, permițând dezvoltatorilor să lucreze cu obiecte Java în loc de instrucțiuni SQL brute. Astfel, se reduce cantitatea de cod necesar pentru a accesa și manipula datele în baza de date.

-> Simplificarea operațiilor CRUD: Data JPA furnizează un set de metode predefinite pentru a realiza operațiile fundamentale CRUD (Create, Read, Update, Delete) asupra entităților din baza de date. Aceasta facilitează crearea de repository-uri și operarea cu datele într-un mod consistent și standardizat.

-> Mapping-ul automat între obiecte Java și tabelele din baza de date: Prin intermediul anotărilor și convențiilor de denumire, Data JPA permite realizarea automată a mapării între obiectele Java și tabelele corespunzătoare din baza de date. Această funcționalitate simplifică dezvoltarea și întreținerea aplicațiilor, eliminând nevoia de a scrie manual cod pentru maparea obiect-relațională.

-> Suportul pentru interogări complexe și specificații: Data JPA oferă un set de funcții și metode pentru a construi interogări complexe asupra datelor, permițând dezvoltatorilor să efectueze operațiuni de filtrare, sortare și agregare a datelor în mod flexibil și eficient.

În concluzie, Data JPA reprezintă o unealtă esențială în arsenalul dezvoltatorilor Java, oferind o abordare simplificată și eficientă pentru lucrul cu bazele de date relaționale în cadrul aplicațiilor Spring. Prin abstractizarea complexității și oferirea unui set de funcționalități puternice, Data JPA contribuie la accelerarea dezvoltării și la îmbunătățirea calității aplicațiilor Java

D. Thymeleaf

Thymeleaf este un motor de șabloane (template engine) care este utilizat în dezvoltarea aplicațiilor web Java. Este integrat strâns cu ecosistemul Spring și oferă o modalitate elegantă de a crea pagini web dinamice și interactivitate în aplicațiile web. Unul dintre aspectele distinctive ale Thymeleaf este abilitatea sa de a combina HTML-ul static cu expresii Thymeleaf, permițând dezvoltatorilor să creeze pagini web care pot fi ușor de modificat și extins folosind sintaxa familiară a HTML-ului. Astfel, Thymeleaf oferă o integrare fără probleme între straturile de prezentare și logică din spatele aplicației.

Principalele caracteristici și beneficii ale utilizării Thymeleaf includ:

-> **Sintaxă prietenoasă cu HTML-ul:** Thymeleaf utilizează o sintaxă familiară cu HTML-ul, ceea ce face mai ușor pentru dezvoltatori să înțeleagă și să modifice paginile web. Expresiile Thymeleaf sunt integrate în mod natural în markup-ul HTML, permițând o separare clară între aspectul vizual și logica aplicației.

-> **Suport pentru expresii și variabile dinamic:** Thymeleaf oferă un set bogat de funcții și expresii care permit afișarea datelor dinamice și evaluarea logică în cadrul paginilor web. Aceasta permite dezvoltatorilor să creeze pagini web dinamice și personalizate în funcție de contextul și datele aplicației.

-> **Integrare strânsă cu Spring:** Thymeleaf este integrat strâns cu Spring Framework, permițând o utilizare simplă și eficientă în cadrul aplicațiilor Spring. Aceasta facilitează transferul datelor între controlerele Spring și șabloanele Thymeleaf, precum și gestionarea evenimentelor și acțiunilor în cadrul paginilor web.

-> **Suport pentru internaționalizare și localizare:** Thymeleaf oferă un suport puternic pentru internaționalizare și localizare, permițând dezvoltatorilor să creeze pagini web care pot fi ușor adaptate pentru diferite limbi și culturi.

În concluzie, Thymeleaf este *o unealtă puternică și versatilă pentru dezvoltarea aplicațiilor web Java*, oferind o integrare simplă cu Spring și facilitând crearea de pagini web dinamice și personalizate. Prin utilizarea sintaxei prietenoase cu HTML-ul și a funcționalităților puternice, Thymeleaf contribuie la accelerarea dezvoltării și la îmbunătățirea experienței utilizatorilor în aplicațiile web Java.

E. Instrument de build: Maven

Maven este un build tool foarte folosit, inventat de Apache Group, pentru ca să publice și să publice multe proiecte în același timp pentru ca să realizeze un project management mult mai eficient și mai bun. Este scris în Java și este folosit pentru ca să construi proiecte în Java, C, Scala, Ruby, etc. Bazat pe Project Object Model(POM), acest tool a făcut viața developerilor de Java mult mai ușoară atunci vorbim de “developing reports, checks build si testing automation setups”. Maven a fost creat pentru a simplifica procesele de build ale Jakarta Tribute. Cuprinde o mulțime de feature-uri folosite ceea ce explică de ce și-a câștigat popularitatea. Build Tool-urile lui ajută la scripting-ul și crearea unor task-uri de o varietate foarte mare.

Build Tool-ul este necesar pentru următoarele procese:

- Generarea codului sursă
- Generarea documentației din codul sursă
- Compilarea codului sursă
- De a împacheta codurile de compilare în fișierele JAR
- Instalarea pachetelor de cod în repository-ul local, server sau în repository-ul central

F. IntelliJ

IntelliJ IDEA este un mediu de dezvoltare integrat (IDE) dezvoltat de JetBrains, care este utilizat pentru dezvoltarea aplicațiilor software în diverse limbaje de programare, inclusiv Java. Este apreciat pentru interfața sa intuitivă, setul puternic de funcționalități și productivitatea sporită pe care o oferă dezvoltatorilor. Unul dintre punctele forte ale IntelliJ IDEA este *suportul extins pentru limbajul de programare Java*. IDE-ul vine cu o gamă largă

de instrumente și funcționalități care facilitează dezvoltarea, de la editarea și refactorizarea codului până la analiza statică, gestionarea dependențelor și debugging-ul.

O altă caracteristică distinctivă a IntelliJ IDEA este puternicul său sistem de integrare cu alte tehnologii și framework-uri populare din ecosistemul Java. De la Spring și Hibernate la Maven și Gradle, IntelliJ oferă suport nativ pentru o varietate de instrumente și tehnologii, permițând dezvoltatorilor să creeze aplicații Java moderne și eficiente. Este recunoscut pentru setul său extins de funcționalități de productivitate, care îmbunătățesc rapiditatea și eficiența dezvoltatorilor. Funcții precum completarea automată a codului, analiza inteligentă a codului, refactorizarea automată și integrarea cu sisteme de control al versiunilor facilitează munca dezvoltatorilor și contribuie la scrierea unui cod de înaltă calitate într-un timp mai scurt.

Deci, IntelliJ IDEA este un instrument esențial pentru dezvoltatorii Java, oferind un mediu puternic și intuitiv pentru dezvoltarea aplicațiilor software. Cu un set larg de funcționalități de productivitate și suport nativ pentru o varietate de tehnologii Java, IntelliJ IDEA contribuie la accelerarea dezvoltării și la îmbunătățirea calității aplicațiilor Java.

G. HTML + CSS

HTML și CSS sunt două tehnologii fundamentale în dezvoltarea paginilor web moderne. HTML (HyperText Markup Language) este limbajul de marcă pentru structurarea conținutului unei pagini web, în timp ce CSS (Cascading Style Sheets) este folosit pentru stilizarea și formatarea acestui conținut.

HTML oferă structura de bază a unei pagini web, permițând dezvoltatorilor să organizeze conținutul în diferite elemente semantice, cum ar fi titluri, paragrafe, liste și imagini. Cu ajutorul etichetelor și atributelor HTML, dezvoltatorii pot crea o structură clară și bine organizată a unei pagini web, care poate fi ușor interpretată și afișată de către browserele web.

Pe de altă parte, **CSS** completează HTML-ul prin oferirea posibilității de a stiliza și formata aspectul vizual al paginii web. Cu ajutorul regulilor CSS, dezvoltatorii pot defini culori, fonturi, dimensiuni, layout-uri și alte proprietăți de stilizare pentru diferitele elemente HTML. Astfel, CSS permite crearea unui aspect atractiv și coeziv pentru o pagină web, contribuind la îmbunătățirea experienței utilizatorilor.

Odată cu avansarea tehnologiilor web, HTML și CSS au evoluat pentru a permite dezvoltarea de pagini web responsive și adaptabile la diferite dispozitive și dimensiuni de ecran. Tehnici moderne cum ar fi media queries, flexbox și grid layout permit dezvoltatorilor să creeze pagini web care se redimensionează și se adaptează în mod dinamic în funcție de mediul de vizualizare.

În concluzie, HTML și CSS sunt două tehnologii esențiale în dezvoltarea paginilor web moderne, oferind o structură semnificativă și un aspect vizual plăcut. Cu ajutorul acestor tehnologii, dezvoltatorii pot crea pagini web atractive, funcționale și adaptabile, care sunt fundamentale pentru prezența și succesul online al unei afaceri sau proiecte.

H. Git (GitHub)

Git și **GitHub** sunt două instrumente esențiale în gestionarea codului sursă în dezvoltarea software și colaborarea între dezvoltatori. Git este un sistem de control al versiunilor distribuit (DVCS) care permite urmărirea modificărilor codului sursă și gestionarea acestora într-un mod eficient și sigur. Pe de altă parte, GitHub este o platformă de găzduire a codului sursă bazată pe web, care facilitează colaborarea și partajarea proiectelor între dezvoltatori din întreaga lume. Unul dintre beneficiile majore ale utilizării Git este capacitatea sa de a urmări istoricul modificărilor codului sursă. Folosind Git, dezvoltatorii pot crea ramuri (branch-uri) separate pentru diferitele funcționalități sau modificări ale proiectului, permițându-le să lucreze în paralel fără a interfera unii cu alții. Astfel, Git facilitează dezvoltarea colaborativă și gestionarea eficientă a fluxului de lucru.

GitHub extinde funcționalitățile Git prin furnizarea unei platforme centralizate pentru găzduirea și colaborarea asupra proiectelor de cod sursă. Pe lângă gestionarea versiunilor și a ramurilor, GitHub oferă funcționalități precum probleme (issues), solicitări de tragere (pull requests), wiki-uri și sisteme de gestionare a proiectelor, care facilitează colaborarea și comunicarea între membrii echipei de dezvoltare. O altă caracteristică importantă a GitHub este capacitatea sa de a oferi un cadru deschis și transparent pentru contribuțiile comunității. Prin intermediul solicitărilor de tragere, orice dezvoltator poate propune modificări la proiectele găzduite pe GitHub și poate colabora cu alți membri ai comunității pentru revizuirea și

integrarea acestora în codul sursă principal. Această abordare deschisă și colaborativă contribuie la dezvoltarea și îmbunătățirea continuă a proiectelor open-source.

În concluzie, Git și GitHub sunt două instrumente esențiale în gestionarea și colaborarea asupra codului sursă în cadrul proiectelor de dezvoltare software. Prin intermediul lor, dezvoltatorii pot lucra eficient, colaborați și îmbunătăți continuu proiectele lor, contribuind la inovație și progres în domeniul tehnologiei.

IV. BIBLIOGRAFIE

- https://ro.wikipedia.org/wiki/Baz%C4%83_de_date#:~:text=O%20baz%C4%83%20de%20date%2C%20uneori,a%20reg%C4%83sirii%20rapide%20a%20acestora. **(Baze de date)**
- <https://www.postgresql.org/about/> **(PostgreSQL)**
- <https://uncoded.ro/structura-limbajului-sql/> **(Limbajul SQL)**
- <https://support.microsoft.com/ro-ro/topic/access-sql-concepte-de-baz%C4%83-vocabular-%C8%99i-sintax%C4%83-444d0303-cde1-424e-9a74-e8dc3e460671> **(Limbajul SQL)**
- [https://en.wikipedia.org/wiki/Java_\(programming_language\)](https://en.wikipedia.org/wiki/Java_(programming_language)) **(Java)**
- https://www.java.com/en/download/help/whatis_java.html **(Java)**
- <https://aws.amazon.com/what-is/java/> **(Java)**
- <https://ramonnastase.ro/blog/programare-orientata-pe-obiecte-poo/> **(Programare orientata pe obiecte)**
- <https://www.link-academy.com/programarea-orientata-pe-obiecte> **(Programare orientata pe obiecte)**
- [Head First Java, 3rd Edition, Kathy Sierra, Bert Bates, Trisha Gee, 2022, O'Reilly Media, Inc, ISBN: 9781491910771](#)
- <https://spring.io/projects/spring-framework> **(Sprig Framework)**
- <https://spring.io/projects/spring-boot> **(Spring Boot)**
- <https://spring.io/projects/spring-data-jpa> **(Data JPA)**
- <https://www.thymeleaf.org/> **(Thymeleaf)**
- <https://docs.spring.io/spring-framework/reference/web/webmvc-view/mvc-thymeleaf.html> **(Thymeleaf)**
- [Maven: The Definitive Guide, Sonoty.pe Comany, 2008, O'Reilly Media Inc, ISBN:97805596551780](#)

- <https://docs.spring.io/spring-framework/reference/web/webmvc-view/mvc-thymeleaf.html> (Instrument de build: Maven)
- https://en.wikipedia.org/wiki/IntelliJ_IDEA (IntelliJ IDEA)
- <https://www.jetbrains.com/help/idea/discover-intellij-idea.html> (IntelliJ IDEA)
- <http://itee.elth.pub.ro/~vbucata/ia/laboratoare/modul2-programare-web/01-introducere.php> (HTML și CSS)
- [HTML & CSS: Design and Build Websites, Jon Duckett, 2011, John Wiley & Sons Inc, ISBN:1118008189](#)
- <https://www.newtech.ro/blog-ce-este-github/> (Github)
- <https://git-scm.com/> (Git)
- <https://www.baeldung.com/hibernate-one-to-many> (One to many JPA)
- <https://www.baeldung.com/spring-boot-h2-database> (h2 Database)
- <https://www.baeldung.com/jpa-one-to-one> (One to one JPA)
- <https://www.baeldung.com/jpa-many-to-many> (Many to many JPA)
- <https://www.baeldung.com/hibernate-one-to-many> (One to many)
- <https://www.cockroachlabs.com/blog/what-is-a-foreign-key/> (Foreign key)
- <https://www.codejava.net/frameworks/spring-boot/spring-boot-thymeleaf-form-handling-tutorial> (Tutorial Formular Thymeleaf)